

# 第 29 章 SELinux 文件规则、端口类型与 Boolean

## 本章阅读导航

---

先抓住一条主线：确认 SELinux 参与拒绝之后，不是立刻“放宽策略”，而是先判断问题属于路径对象、协议端口还是策略预留的可选行为，再建立持久声明并用分层证据验收。

- 01 识别对象路径、协议端口或可选行为
- 02 建立基线规则、预期、当前与功能分开
- 03 选择入口 fcontext、port type 或 Boolean
- 04 最小修改新增、修改或删除已有声明
- 05 应用当前状态让持久声明作用于现有对象
- 06 分层验收配置、运行、网络、SELinux 与功能

## 专题地图

- 知识专题文件、端口和行为：先选对策略入口
- 知识专题当前标签、预期标签与持久规则
- 操作专题建立文件上下文基线并安全恢复标签
- 操作专题管理 fcontext 规则生命周期
- 知识专题文件类型与最小写能力
- 操作专题管理协议相关的非标准端口
- 操作专题查询并持久设置 Boolean
- 诊断专题规则覆盖、端口冲突与跨层证据
- 经典任务自定义目录和 TCP 82 Web 服务
- 参考解答调查、修改、应用和分层验收

## 阅读时持续回答

1. 现在看到的是持久规则、预期标签还是当前标签？
2. 目标表达式是否同时覆盖目录本身和后代？
3. 已有记录应使用新增、修改还是删除？
4. 端口号对应的是 TCP 还是 UDP？
5. Boolean 是否精确描述真实业务行为？
6. 这条验证命令能证明什么，不能证明什么？
7. 最终成功是否仍依赖临时标签或 runtime-only 状态？

判断提示：每次变更都先区分持久声明、当前状态与最终功能；阅读经典任务时先独立列出验收矩阵，再对照参考解答。

## 第 29 章 · 正文

上一章已经建立 SELinux 模式、进程域、对象类型和 AVC 证据的最小判断框架。本章从“已经确认 SELinux 参与拒绝”这一节点继续：面对自定义目录、非标准端口或策略默认关闭的业务行为，怎样选择正确的标准策略入口，怎样区分当前状态与持久声明，并用最小授权完成可重复的验收。

最常见的误判，是把一次可见变化当作长期正确状态：``chcon`` 后 ``ls -Z`` 看起来正确，却没有路径规则；``semanage port`` 成功，却没有进程监听；Boolean 为 ``on``，却没有重新触发真实应用。为避免这些局部成功，本章始终沿着“对象 - 声明 - 当前状态 - 功能证据”推进。

SELinux 模型与 AVC 解读归第 28 章；firewalld 规则归第 21 章；服务 unit 与启动状态归第 12 章。本章只在综合验收时引用这些证据入口，不重复展开相邻章节。

### 概念

当前标签 是文件或目录此刻携带的 SELinux 安全上下文，可由 `ls -Z` 观察。它可能来自创建继承、`restorecon`、`chcon` 或文件移动等过程，因此“当前看起来正确”只证明这一时刻的对象状态，不能证明系统已经建立路径级持久声明。

### 概念

持久 fcontext 规则 是管理员登记的“路径表达式到 SELinux 类型”的映射。它描述该路径未来应获得什么默认类型，却不会在创建或修改规则时自动遍历磁盘对象；已有文件仍需由 `restorecon` 按规则应用。规则存在、预期正确和当前标签正确必须分别验证。

### 概念

路径正则 是 fcontext 规则识别对象的表达方式。经典形式 `'/srv/site(/.*)?'` 同时覆盖目录本身和斜杠后的所有后代；外层单引号防止 Shell 把括号、星号和问号当作自己的语法。表达式过宽会扩大影响面，过窄则可能漏掉目录或内容。

### 概念

端口类型 把“协议 + 端口或端口范围”映射为 SELinux 策略对象，决定某类受限进程是否能使用该网络端口。TCP 82 与 UDP 82 是不同对象；端口类型正确只证明 SELinux 的一个条件，不证明应用配置、真实监听或 firewalld 已经正确。

### 概念

Boolean 是策略作者预留的条件开关，用于整体启用或关闭一组已定义规则，例如允许 Web 应用主动连接数据库。它不负责修复错误文件标签、错误端口映射或 DAC 权限；选择 Boolean 时必须从真实业务行为出发，并区分当前值与使用 `-P` 保存的持久值。

### 概念

最小授权 是只改变能够解释当前证据的最窄状态：静态内容使用只读类型，写入能力只授予必要子目录；端口映射同时限定协议和端口；Boolean 只启用语义匹配的开关。关闭 SELinux、`chmod 777`、批量可写类型或盲目生成策略模块都不是默认修复。

操作语义以下命令分别维护路径规则、计算预期标签、应用当前标签、管理协议端口和切换策略 Boolean。先理解作用对象，再选择参数。

## semanage fcontext

### SYNOPSIS

```
semanage fcontext {-l|-a|-m|-d} [-t TYPE] FILE_SPEC
```

查询或维护持久的路径表达式到文件类型映射；它修改规则，不直接重标记现有文件。

### 重要参数 / 形式

**-l / -C -l**

列出全部规则，或只列出本地定制。

**-a -t TYPE FILE\_SPEC**

在表达式不存在时新增规则。

**-m -t TYPE FILE\_SPEC**

修改已经存在且确认需要调整的规则。

**-d FILE\_SPEC**

删除本地规则；删除后不会自动改变磁盘上的当前标签。

## matchpathcon / restorecon

### SYNOPSIS

```
matchpathcon [-V] PATH
restorecon [-n] [-R] [-v] PATH ...
```

前者根据当前规则计算路径的预期上下文；后者把该预期应用到现有对象。

### 重要参数 / 形式

**matchpathcon PATH**

显示路径按规则应获得的默认上下文。

**matchpathcon -V PATH**

比较预期上下文与磁盘当前上下文。

**restorecon -nRv PATH**

递归预览将发生的变化，不实际修改。

**restorecon -Rv PATH**

递归应用当前文件上下文规则。

## chcon

### SYNOPSIS

```
chcon -t TYPE PATH ...
```

直接修改对象当前标签，适合受控实验或临时定位；它不建立路径未来应获得什么类型的持久规则。

重要参数 / 形式

`-t TYPE`

只替换上下文中的类型字段。

`chcon -t TYPE PATH`

当前标签可跨普通重启保留，但可能被 `restorecon`、重标记或对象重建覆盖。

**semanage port**

SYNOPSIS

```
semanage port {-l|-a|-m|-d} [-t TYPE] -p {tcp|udp} PORT
```

查询或维护 SELinux 的协议相关端口映射；端口号不能脱离 TCP 或 UDP 单独判断。

重要参数 / 形式

`-l / -C -l`

列出全部端口映射，或只列出本地定制。

`-p tcp|udp`

明确协议；同一数字的 TCP 和 UDP 是不同对象。

`-a/-m/-d`

分别用于新增、修改和删除本地端口映射。

`-t TYPE`

指定目标 SELinux 端口类型。

**getsebool / setsebool**

SYNOPSIS

```
getsebool {NAME|-a}
setsebool [-P] NAME {on|off}
```

查询当前 Boolean，并切换策略预先定义的可选行为；选择开关前先阅读名称和描述是否精确匹配业务。

重要参数 / 形式

`getsebool NAME`

查询指定 Boolean 当前值。

`getsebool -a`

列出全部当前值，通常配合筛选发现相关项。

`setsebool NAME on|off`

只改变当前加载策略中的值。

`setsebool -P NAME on|off`

同时建立持久值；等待命令完成后再验证。

## [知识专题] 文件、端口和行为：先选对策略入口

SELinux 拒绝本身不是修复答案。相同的“服务不可用”现象可能来自目标文件类型不合适、服务绑定了策略未授权的端口，或者服务确实需要策略默认关闭的一项可选行为。最快的切入方法不是依次尝试命令，而是先问：失败动作主要落在路径对象、网络端口，还是策略预留的业务行为上。

### ① [知识点] 文件上下文规则管理路径对象的用途

受限进程不是按传统文件名猜测对象用途，而是根据目标对象类型和请求行为执行策略规则。将站点从 `/var/www/html` 移到 `/srv/site` 后，即使普通权限允许读取，文件也可能仍带有与 Web 内容不相符的类型。此时应建立路径的持久 `fcontext` 规则，并把规则应用到当前对象。

典型闭环是：

```
持久规则: semanage fcontext
          ↓
预期标签: matchpathcon
          ↓
当前标签: restorecon 应用, ls -Z 查看
          ↓
真实访问: 重新请求服务功能
```

### ② [知识点] 端口类型管理受限服务可使用的网络端口

服务配置文件决定应用计划监听什么端口；SELinux 端口类型决定相应进程域是否被策略允许绑定该协议和端口。两者不一致时，服务可能在启动或重载时因 `name_bind` 被拒绝。

端口映射不能脱离协议：

```
(http_port_t, tcp, 82)
```

与 UDP 82 不是同一条记录。端口可能已经属于其他类型或位于一个已有范围中，所以操作前应先查，不应把 `-a` 当作固定答案。

### ③ [知识点] Boolean 管理策略已经预留的可选行为

Boolean 不直接给某个具体文件贴标签，也不把某个端口改成服务端口。它一次启用或关闭策略中围绕某项业务行为准备好的一组条件规则。例如 Web 应用确实需要主动连接数据库时，可以调查 `httpd_can_network_connect_db`；如果只是站点目录标签错误，开启该 Boolean 没有帮助。

选择标准入口的快速判断：

读取、写入或搜索某个路径失败  
→ 检查当前标签、预期标签和 `fcontext` 规则

服务绑定非标准端口失败  
→ 检查协议、端口及其 SELinux 端口类型

合法业务行为被默认策略关闭  
→ 搜索并理解对应 Boolean

#### ④ [边界] 标准入口优先于扩大权限和自定义策略

遇到拒绝时，先排除错误配置，再检查 SELinux 已经提供的文件类型、端口类型和 Boolean。以下做法不能作为默认答案：

- 关闭 SELinux 或长期保持 Permissive；
- 使用 `chmod 777` 试图绕过 SELinux；
- 将整个业务目录都标记成可写类型；
- 看见 AVC 后立即用 `audit2allow` 生成本地模块；
- 不理解含义就批量开启名称相近的 Boolean。

只有标准策略确实无法表达经过确认的合法需求时，才进入自定义策略设计；这属于更高级的 SELinux 工作，不在本章完整展开。

[Cheatsheet] 路径对象看 `fcontext`，协议端口看 `port type`，可选行为看 Boolean；先修误配置，再考虑扩展策略。

## [知识专题] 当前标签、预期标签与持久规则是三个状态

文件标签问题最容易出现“看起来已经修好”的误判。`ls -Z` 显示正确，只证明此刻对象标签正确；它不能证明系统存在持久路径规则。反过来，`semanage fcontext` 已经添加规则，也不能证明磁盘上已有文件已经被重新标记。稳定判断必须把规则、预期和当前三个状态分开。

### ① [知识点] `ls -Z` 读取当前对象标签

常用形式：

```
ls -Zd /srv/site
ls -lZ /srv/site
```

目录本身和目录中的文件是不同对象。只执行 `ls -lZ /srv/site` 时，容易只看到内容而忽略目录自身，因此调查路径穿越或目录类型时应显式查看目录本身。

当前标签可能来自：

- 创建文件时根据父目录和策略获得；

- `restorecon` 按默认规则恢复；
- `chcon` 直接修改；
- 复制、移动、解压或应用创建文件时的具体行为；
- 文件系统重标记。

所以当前值本身不能回答它是否会在下一次恢复标签后继续存在。

② [知识点] `matchpathcon` 读取路径的预期标签

```
matchpathcon /srv/site/index.html
matchpathcon -V /srv/site/index.html
```

第一条命令根据当前加载的文件上下文规则计算该路径的默认上下文。`-V` 用于比较磁盘上的当前标签与默认值。该证据回答的是“策略认为此路径应该是什么”，而不是“服务是否一定有权执行某个具体操作”。

如果 `matchpathcon` 给出的预期类型本身就不符合目标，继续重复 `restorecon` 没有意义，因为 `restorecon` 只会忠实应用当前规则。下一步应检查本地 `fcontext` 定制和规则重叠。

③ [知识点] `semanage fcontext` 读取和维护持久规则

```
semanage fcontext -l
semanage fcontext -C -l
```

`-l` 列出策略提供的规则和本地定制；`-C -l` 只列出本地自定义记录，适合定位管理员覆盖。持久规则维护的是路径表达式，不直接遍历并修改现有文件。

稳定闭环应分别证明：

| 状态     | 推荐证据                                                       | 证明范围              |
|--------|------------------------------------------------------------|-------------------|
| 持久路径规则 | <code>semanage fcontext -C -l</code> 或精确筛选 <code>-l</code> | 管理员定义了怎样的路径映射     |
| 预期标签   | <code>matchpathcon</code>                                  | 当前规则为该路径计算出的默认上下文 |
| 当前标签   | <code>ls -Z</code>                                         | inode 此刻携带的标签     |
| 应用功能   | 服务请求或读写测试                                                  | 目标业务行为是否真正完成      |

④ [边界] `chcon` 不是“重启即失效”，但仍不是持久规则

`chcon -t TYPE PATH` 直接改变对象当前标签。这个标签通常可以跨普通重启保留，因此把 `chcon` 简化成“重启后必定失效”并不准确。真正的边界是：它没有建立路径级默认规则，后续 `restorecon`、文件系统重新标记、对象重建或其他标签恢复过程可以覆盖它。

因此 `chcon` 可以用于受控实验或临时定位，但考试和稳定运维的最终配置应回到：

```
semanage fcontext 建规则
→ restorecon 应用规则
→ matchpathcon 与 ls -Z 比较
→ 功能验证
```

[Cheatsheet] 当前看 `ls -Z`，预期看 `matchpathcon`，持久规则看 `semanage fcontext`；`chcon` 改当前对象，不定义路径未来应该是什么。

## [操作专题] 建立文件上下文基线并安全执行 `restorecon`

在修改路径规则前，先记录规则、预期和当前状态。这样可以判断故障来自规则缺失、当前标签漂移，还是服务需要的类型选择本身错误，也能避免在包含现有业务数据的目录上盲目递归重标记。

### ① [操作] 对一个路径建立三层基线

作用对象：目标目录本身、关键文件以及它们匹配的 `fcontext` 规则。

```
ls -Zd /srv/site
ls -lZ /srv/site
matchpathcon /srv/site
matchpathcon /srv/site/index.html
semanage fcontext -C -l | grep -F '/srv/site'
```

若本地列表没有结果，不代表策略完全没有默认规则；还应按需检查完整列表或直接以 `matchpathcon` 的结果作为“当前规则最终推导出的预期值”。使用 `grep` 只是缩小输出，不要因为正则符号被 `grep` 解释而误判记录不存在；需要精确查看时应保留完整行。

### ② [操作] 先预览再递归应用

```
restorecon -nRv /srv/site
restorecon -Rv /srv/site
```

关键参数：

- `-n`：只显示计划变更，不真正修改；
- `-R`：递归处理目录树；
- `-v`：显示处理信息。

在真实服务器上，递归操作前先用 `-n` 预览更稳妥，特别是目录中可能混有上传区、缓存、密钥或其他用途不同的子目录时。预览发现大量意外变化，应先调整规则边界，而不是直接执行。

### ③ [操作] 应用后比较规则、预期与当前

```
matchpathcon -V /srv/site
matchpathcon -V /srv/site/index.html
ls -Zd /srv/site
ls -lZ /srv/site
```

标签一致仍只是 SELinux 对象状态的一部分。随后必须重新触发服务读取、写入或执行操作，并查看最新失败证据。旧 AVC 不能证明当前修复仍失败。

### ④ [边界] `restorecon` 没有输出不等于“命令没有工作”

目标本来就符合预期时，普通或非详细模式可能没有需要报告的变化。判断应基于：

1. `matchpathcon` 给出的预期；
2. `ls -Z` 给出的当前；
3. `restorecon -nRv` 是否计划变化；
4. 重新触发后的实际功能。

不要为了获得“有输出”而反复执行 `chcon` 制造差异。

[Cheatsheet] 基线先分规则、预期、当前；递归恢复先 `-nRv` 预览，再 `-Rv` 应用；最后重新触发原始业务。

## [操作专题] 用 `semanage fcontext -a/-m/-d` 管理规则生命周期

`semanage fcontext` 维护的是本地策略存储中的文件上下文定制。新增、修改和删除对应不同的现状，不能把 `-a` 重复执行当作幂等操作。考试环境常见的错误是规则已经存在，但类型错误；真实工作中还常见旧规则过宽、多个表达式重叠或删除规则后没有重新恢复当前标签。

### ① [操作] `-a` 只用于新增不存在的规则

```
semanage fcontext -a -t httpd_sys_content_t '/srv/site(/.*)?'
restorecon -Rv /srv/site
```

作用对象：路径表达式 `'/srv/site(/.*)?'` 的持久默认类型。

`-t` 指定目标类型。表达式使用单引号，避免 Shell 把括号、问号或星号解释为自己的语法。`(/.*)?` 的直观作用是：既匹配 `/srv/site` 目录本身，也匹配斜杠之后的所有后代路径。

只给 `/srv/site` 建一条不含后代的规则，通常不能覆盖目录中的文件；只给 `/srv/site/.*` 建规则又可能漏掉目录本身。经典形式同时覆盖二者。

## ② [操作] 已有规则类型不符时使用 `-m`

```
semanage fcontext -m -t httpd_sys_content_t '/srv/site(/.*)?'
restorecon -Rv /srv/site
```

判断过程：

```
semanage fcontext -a 报记录已存在
→ 查询本地和完整规则
→ 确认表达式就是目标记录
→ 判断原类型是否确实错误
→ 使用 -m 修改
→ restorecon 应用
```

不要因为 `-a` 报错就直接改写一个看不懂的现有规则。表达式可能属于已有业务，或者目标路径实际被另一条更高优先级本地规则命中。

## ③ [操作] 不再需要本地定制时使用 `-d`

```
semanage fcontext -d '/srv/site(/.*)?'
matchpathcon /srv/site/index.html
restorecon -nRv /srv/site
restorecon -Rv /srv/site
```

删除操作只移除本地规则，不会自动把磁盘上已经存在的标签改回策略默认值。应先重新查询删除后的预期标签，再预览和应用 `restorecon`。如果删除后路径不再有适合业务用途的默认类型，服务可能重新失败，因此删除也必须经过功能验证。

## ④ [配置] 本地规则优先于策略包规则

管理员通过 `semanage fcontext` 添加的本地定制用于覆盖或补充策略包的默认文件上下文。它们应被视为本机配置资产，不能直接手工编辑底层数据库文件代替 `semanage`。

本地表达式发生重叠时，不能只凭“看起来更具体”猜谁生效。当前工具语义下，本地规则按较新的记录优先匹配，第一条匹配项决定预期上下文。稳定方法是直接对具体路径执行 `matchpathcon`，并通过 `semanage fcontext -C -l` 检查本地记录和新增顺序造成的覆盖。

## ⑤ [配置] 规则应尽量窄并按用途拆分

假设站点大部分内容只读，但 `/srv/site/uploads` 确实需要 Web 进程写入，应将范围拆开：

```
semanage fcontext -a -t httpd_sys_content_t '/srv/site(/.*)?'
semanage fcontext -a -t httpd_sys_rw_content_t '/srv/site/uploads(/.*)?'
restorecon -Rv /srv/site
```

然后用 `matchpathcon` 分别检查普通页面和上传目录。真实环境中还应留意本地规则的覆盖顺序，确保窄规则真正成为最终匹配；发现预期不符时，调整规则而不是给整个站点扩大写类型。

## ⑥ [帮助入口] 工具缺失和语法不确定时从本地帮助恢复

```
semanage fcontext -h
man semanage-fcontext
man restorecon
man matchpathcon
dnf provides '*/semanage'
```

`semanage` 不存在时，先检查拼写和已启用仓库，再用 `dnf provides` 查询当前系统中由哪个包提供命令，不必依赖脱离环境的包名记忆。

[Cheatsheet] 不存在才 `-a`，已存在且确需改才 `-m`，移除本地覆盖用 `-d`；每次规则变化后重新计算预期，再决定 `restorecon`。

## [知识专题] 文件类型代表能力边界，可写类型必须最小化

给文件选择 SELinux 类型，不是给路径添加一个“允许访问”的通用标志，而是在声明对象用途。不同类型让策略允许不同域执行不同操作。类型选择过窄会阻止合法业务，选择过宽则会扩大被入侵进程可以影响的数据范围。

### ① [知识点] `httpd_sys_content_t` 适合 Web 只读内容

静态页面、样式文件和只需要被 Web 服务读取的资源，典型目标类型是 `httpd_sys_content_t`。它表达“这是 Web 内容”，不表示所有进程都能访问，也不改变普通文件权限。

### ② [知识点] `httpd_sys_rw_content_t` 只用于确实需要写入的对象

上传目录、应用缓存或会话目录可能需要 Web 进程写入。只有经过业务确认的最小子目录才应使用可写类型。把整个 `/srv/site` 标成可写类型虽然可能让错误暂时消失，却扩大了 Web 进程在被攻陷后可修改的内容。

### ③ [知识点] SELinux 类型不会绕过 DAC

目录已经标记为可写类型，不代表 `httpd` 一定能写入。路径上每一级目录的搜索权限、目标目录的所有者和模式、ACL 等 DAC 条件仍然必须允许。反过来，`chmod 777` 也不会修复不匹配的 SELinux 类型。

建议的分层证据：

```
namei -l /srv/site/uploads/file.dat
ls -Zd /srv/site/uploads
matchpathcon /srv/site/uploads
```

#### ④ [边界] 不要从服务名称机械推导类型

策略包可能为同一服务提供多个用途不同的类型，名称也可能随策略更新。选择前应查看服务对应的 SELinux 手册页、现有默认目录标签或官方文档，例如：

```
man httpd_selinux
matchpathcon /var/www/html
```

考试题的典型类型可以记忆，但真实工作中仍应以目标系统已安装策略为准。

[Cheatsheet] 类型声明用途；静态内容只读，写入目录单独划分；DAC 和 SELinux 必须同时允许。

## [操作专题] 为非标准端口建立协议相关的 SELinux 类型

服务监听非标准端口时，应同时管理四个独立状态：应用配置中的端口、SELinux 端口类型、进程真实监听以及 firewalld 的网络放行。只修改其中一项不会自动同步其他层。

#### ① [操作] 先查询类型、协议、端口和本地定制

```
semanage port -l | grep -w http_port_t
semanage port -C -l
```

第一条按类型查看策略允许的端口集合，第二条只列本地定制。还应检查目标端口是否已属于其他类型或端口范围。筛选输出时同时关注协议列，不能因为数字相同就把 TCP 和 UDP 记录混为一谈。

#### ② [操作] 目标端口没有记录时使用 `-a`

```
semanage port -a -t http_port_t -p tcp 82
```

参数语义：

- `-a`：添加一条本地端口映射；
- `-t http_port_t`：指定目标 SELinux 端口类型；
- `-p tcp`：指定协议；
- `82`：端口号，也可以是命令支持的端口范围形式。

命令成功只说明本地策略存储接受了映射，不说明 `httpd` 配置正确或已经监听。

#### ③ [操作] 已有记录且确需改变类型时使用 `-m`

```
semanage port -m -t http_port_t -p tcp 82
```

使用前必须确认：

1. TCP 82 当前确实已有记录；
2. 该记录不是另一个合法服务所需；
3. 题目或业务目标明确要求由 httpd 使用；
4. 修改不会破坏同机其他服务。

如果端口位于一个范围记录中，应先调查精确记录和影响面，不要假设修改单个数字一定是正确操作。

#### ④ [操作] 删除不再需要的本地端口映射

```
semanage port -d -p tcp 82
```

删除后应重新查询端口列表，并确认没有仍在使用该映射的服务。删除 SELinux 映射不会自动修改应用配置、停止监听或移除防火墙规则，清理需要逐层处理。

#### ⑤ [验证] 把五层证据分开

```
# 应用配置（示例为 Apache）
grep -R '^[:space:]*Listen' /etc/httpd/conf /etc/httpd/conf.d

# SELinux 端口类型
semanage port -l | grep -w http_port_t

# 服务当前与持久状态
systemctl is-active httpd.service
systemctl is-enabled httpd.service

# 真实监听
ss -lntp | grep ':82'

# firewallld 与最终功能
firewall-cmd --list-ports
curl http://localhost:82/
```

这些命令来自不同章节，本章只使用它们建立验收矩阵，不重新展开 Apache、systemd 或 firewallld 的完整机制。

[Cheatsheet] 端口身份是协议加数字；先查再 `-a/-m`；端口类型正确后仍要验证配置、服务、监听、防火墙和实际请求。

## [操作专题] 从业务行为找到并持久设置 Boolean

Boolean 的关键不是记住尽可能多的名称，而是从受限进程正在尝试的合法业务行为出发，找到策略已经预留的精确开关，理解它会扩大哪类能力，然后建立当前和持久终态。

## ① [操作] 查询当前值并搜索相关开关

```
getsebool httpd_can_network_connect_db
getsebool -a | grep '^httpd_'
semanage boolean -l | grep -i httpd
```

`getsebool` 直接读取当前已加载策略中的值；`semanage boolean -l` 提供更完整的列表和描述，有助于区分名称相近但行为范围不同的开关。搜索只用于发现相关项，最终选择要回到业务语义。

## ② [操作] 不带 `-P` 只改变当前值

```
setsebool httpd_can_network_connect_db on
```

这种形式适合明确知道需要临时验证当前行为的场景。它不能证明重启或策略重新加载后仍保持。测试后应根据目标决定持久化或恢复原值，并重新触发原始业务。

## ③ [操作] 使用 `-P` 建立持久值

```
setsebool -P httpd_can_network_connect_db on
getsebool httpd_can_network_connect_db
```

`-P` 将设置写入持久策略配置，处理可能比普通当前值修改更慢。命令尚未返回时不要重复执行；完成后再查询当前值，并在后续真实环境验证重启或策略重新加载后的状态。

## ④ [判断] Boolean 必须与真实业务行为匹配

以下判断不是同一件事：

- Web 服务读取本地静态目录：优先检查文件类型；
- Web 服务绑定 TCP 82：优先检查端口类型；
- Web 应用主动连接 MariaDB：调查数据库网络连接 Boolean；
- Web 服务读取 NFS 上的内容：调查与网络文件系统相关的 Boolean 和挂载状态。

不要为了“让它能连出去”启用范围更宽的开关，除非业务确实需要更宽能力并经过风险评估。

## ⑤ [验证] 当前值正确不等于业务必然成功

Boolean 只控制策略条件。数据库地址、DNS、路由、防火墙、数据库账号、应用配置和服务状态仍然可能失败。Boolean 修改后应重新触发原始应用请求，并以最新日志和功能结果判断。

[Cheatsheet] 从业务行为搜索 Boolean；先读说明再设置；持久终态用 `-P`；当前值正确后仍需重做真实功能。

## [诊断专题] restorecon 后标签仍不符合预期

文件上下文诊断的核心是判断“恢复工具没有执行”“当前标签没有变化”还是“规则本身给出了不正确的预期”。重复运行命令只能验证同一事实，下一条证据应能区分这些假设。

### ① [诊断] 症状：restorecon 没有改变标签

证据链：

```
症状：执行后 ls -Z 看起来不变
→ 当前证据：matchpathcon 与 ls -Z 是否本来就一致
→ 假设 A：对象已经符合预期
→ 假设 B：检查了目录内容，却没有检查目录本身
→ 假设 C：没有递归处理需要改变的后代对象
→ 下一证据：restorecon -nRv + matchpathcon -V
→ 最小修复：只对确认的目标范围应用 -Rv
→ 再验证：当前标签和原始业务
```

### ② [诊断] 症状：恢复后仍得到“错误”的类型

如果 restorecon 把对象改成了某个类型，但该类型不符合业务目标，说明工具正在按规则工作。下一步不是 chcon，而是：

```
matchpathcon /srv/site/index.html
semanage fcontext -C -l
```

检查是否存在旧本地规则、过宽表达式或后添加的重叠规则。修正规则后再应用 restorecon。

### ③ [诊断] 症状：新增窄规则后仍被宽规则覆盖

本地规则重叠时，以具体路径的 matchpathcon 结果为准。先确认哪条本地记录最后匹配，再通过修改、删除并重新添加必要规则调整优先关系。不要只根据正则字符串长度判断。

修复后至少检查两个代表性对象：

```
matchpathcon /srv/site/index.html
matchpathcon /srv/site/uploads/test.dat
```

确保只读区域和可写区域分别得到目标类型。

### ④ [诊断] 症状：删除规则后磁盘标签没有自动回退

semanage fcontext -d 只删除规则。随后执行：

```
matchpathcon /srv/site/index.html
restorecon -nRv /srv/site
```

如果预期回到了策略默认值，再执行最小范围的 `restorecon`。删除前后都应记录功能影响，避免误删现有业务所需的本地规则。

## ⑤ [诊断] 症状：文件移动后保留来源标签

不同文件操作对标签的影响不同。一个文件从用户家目录移动到 Web 目录后仍携带 `user_home_t` 时，优先按目标路径恢复：

```
restorecon -v /var/www/html/<FILE>
```

若目标目录本身是标准路径，通常不需要为单个文件创建新的 `fcontext` 规则。先使用已有默认规则，避免无意义的本地定制。

## ⑥ [安全边界] 变更前控制递归范围

在 `/srv` 或应用数据根目录上直接执行大范围 `restorecon -R` 可能改变多个业务对象。先用 `findmnt` 确认挂载边界、用 `restorecon -nRv` 预览，并把操作收缩到题目要求的目录。不要为了省步骤对整个系统执行无调查重标记。

[Cheatsheet] `restorecon` 服从规则；结果不对先查预期和本地覆盖；删除规则不会自动修改磁盘标签；递归前先预览范围。

# [诊断专题] 端口已有类型、Boolean 无效与跨层证据推进

SELinux 状态正确但业务仍失败并不矛盾。文件规则、端口类型和 Boolean 只是完整访问链中的几个条件。高质量诊断应让每条命令排除一个假设，而不是看到局部成功就宣布完成。

## ① [诊断] `semanage port -a` 报端口已定义

症状：新增失败

- 当前证据：协议、端口、现有类型或范围
- 假设：端口已属于其他类型，或已位于范围记录
- 下一证据：`semanage port -l / -C -l`
- 最小修复：确认无业务冲突后使用 `-m`，或保留原映射并改服务端口
- 再验证：端口类型、服务启动、真实监听

不要无条件删除已有记录再重建；这可能破坏另一个服务的合法映射。

## ② [诊断] 端口类型正确但服务仍无法监听

继续检查：

- 应用配置语法是否正确；
- 端口是否已被其他进程占用；
- 服务日志中的当前错误是否仍是 SELinux 拒绝；

- 配置是否被其他文件覆盖；
- 服务是否实际重启或重载了新配置。

`semanage port` 成功不能证明任何进程已经监听。

### ③ [诊断] Boolean 已为 on，但业务仍失败

重新触发失败并取得当前证据。常见分支包括：

目标文件类型错误  
目标端口类型错误  
DAC 不允许  
应用地址或凭据错误  
网络或远端服务不可达  
选择的 Boolean 与行为不匹配

不要继续开启更多 Boolean 试错。Boolean 的名字相似不代表作用相同。

### ④ [诊断] 防火墙放行、服务 active，但外部访问失败

建立独立证据矩阵：

| 层         | 代表证据                              | 只能证明           |
|-----------|-----------------------------------|----------------|
| 服务配置      | 语法检查、配置文件                         | 应用计划采用的参数      |
| 服务当前状态    | <code>systemctl is-active</code>  | unit 当前运行状态    |
| 持久启动      | <code>systemctl is-enabled</code> | 重启时的启动配置       |
| 监听        | <code>ss -lntp</code>             | 本机 socket 当前存在 |
| firewalld | zone/端口查询                         | 网络过滤层允许        |
| SELinux   | fcontext、port、Boolean             | 强制策略相关条件       |
| 功能        | 本机与远端协议请求                         | 业务路径实际完成       |

一层成功不能替代其他层。

### ⑤ [诊断] 当前访问成功但担心重启后失效

至少检查：

- fcontext 本地规则存在；
- `matchpathcon` 与当前标签一致；
- 端口映射是本地持久记录；
- Boolean 使用了持久设置；
- 服务为 enabled；

- `firewalld` 修改使用了持久配置；
- 再次执行 `restorecon` 后功能仍正常。

本会话没有可控 RHEL 9 VM，重启级验收只作为推荐验证写入质量报告，不声明已完成。

## ⑥ [安全边界] 只修改能够解释当前拒绝的最小状态

修复前写出假设：“因为目标路径预期类型错误，所以修改 `fcontext`”“因为 TCP 82 已有不匹配类型，所以修改该端口映射”“因为业务确实需要 `httpd` 连接数据库，所以启用精确 `Boolean`”。如果不能把修改与证据连接起来，应继续调查。

[Cheatsheet] 新增失败先查现有记录；SELinux 状态正确后继续沿服务、监听、网络和应用层推进；每次只做能解释证据的最小修改。

# [经典任务] 在自定义目录和 TCP 82 上发布 Web 服务

## 环境

一台 RHEL 9 主机已经安装 `httpd`。管理员准备将现有静态站点放在 `/srv/exam-site`，并让 Apache 监听 TCP 82。SELinux 处于 Enforcing；`firewalld` 正在运行。`/srv/exam-site/index.html` 已经存在，禁止删除、重建或改写其内容。

当前状态并不完全已知：

- Apache 配置可能已经改为 TCP 82，也可能存在语法错误；
- 目录和文件当前标签可能由 `chcon` 临时修改过；
- 系统可能存在旧 `fcontext` 或端口本地定制；
- TCP 82 可能尚未映射，也可能已属于其他类型；
- 服务当前和开机状态未知。

## 目标终态

1. `/srv/exam-site` 及其后代拥有持久、最小的 Web 只读文件上下文规则；
2. 当前目录和文件标签与预期规则一致；
3. TCP 82 按 TCP 协议属于适合 `httpd` 的端口类型；
4. `httpd.service` 当前运行，并配置为系统启动时自动启动；
5. 进程实际监听 TCP 82；
6. `firewalld` 允许必要的 TCP 82 访问；
7. 本机及题目指定客户端能够读取现有首页；
8. 再次执行 `restorecon -Rv /srv/exam-site` 后，标签和访问仍正确。

## 限制

- 不关闭或长期放宽 SELinux；

- 不使用 `chcon` 作为最终持久配置；
- 不给整个站点使用可写类型；
- 不生成自定义 SELinux 策略模块；
- 不删除现有端口映射后盲目重建；
- 不把 `active`、监听存在或单次 `curl` 成功单独当作完整终态。

### 验收证据

| 目标      | 推荐证据                                                       |
|---------|------------------------------------------------------------|
| 规则存在    | <code>semanage fcontext -C -l</code>                       |
| 预期标签    | <code>matchpathcon</code>                                  |
| 当前标签    | <code>ls -Z</code>                                         |
| 端口类型    | <code>semanage port -l</code>                              |
| 当前/持久服务 | <code>systemctl is-active</code> 、 <code>is-enabled</code> |
| 真实监听    | <code>ss -lntp</code>                                      |
| 防火墙     | 永久和当前规则查询                                                  |
| 最终功能    | 本机与客户端 <code>curl</code>                                   |
| 抗临时标签   | 再次 <code>restorecon</code> 后重复访问                           |

## [参考解答] 自定义目录和 TCP 82 Web 服务

参考解答展示的是证据顺序，不是假设所有机器都处于同一初始状态。看到已有规则时必须根据实际查询选择 `-m`，不能机械复制 `-a`。

### ① 调查服务配置、当前状态和普通权限

```
httpd -t
grep -R '^[[[:space:]]*Listen' /etc/httpd/conf /etc/httpd/conf.d
systemctl status httpd.service --no-pager
namei -l /srv/exam-site/index.html
ls -Zd /srv/exam-site
ls -lZ /srv/exam-site
```

先修正题目范围内的 Apache 语法错误和普通路径权限。不要用扩大到 `777` 的方式“排除权限”；只确保服务能够穿越路径并读取现有文件。

## ② 调查文件规则、预期和当前标签

```
semanage fcontext -C -l | grep -F '/srv/exam-site'  
matchpathcon /srv/exam-site  
matchpathcon /srv/exam-site/index.html  
restorecon -nRv /srv/exam-site
```

若目标表达式不存在，新增：

```
semanage fcontext -a -t httpd_sys_content_t '/srv/exam-site(/.*)?'
```

若同一表达式已经存在但类型错误，确认后修改：

```
semanage fcontext -m -t httpd_sys_content_t '/srv/exam-site(/.*)?'
```

应用并检查：

```
restorecon -Rv /srv/exam-site  
matchpathcon -V /srv/exam-site  
matchpathcon -V /srv/exam-site/index.html  
ls -Zd /srv/exam-site  
ls -lZ /srv/exam-site
```

## ③ 调查并配置 TCP 82 的端口类型

```
semanage port -l | grep -E '^(^|[:space:]))82([,[:space:]-]|$)'  
semanage port -l | grep -w http_port_t  
semanage port -C -l
```

若 TCP 82 没有任何映射：

```
semanage port -a -t http_port_t -p tcp 82
```

若 TCP 82 已有记录且确认应改由 httpd 使用、不会破坏其他服务：

```
semanage port -m -t http_port_t -p tcp 82
```

再次查询，确认协议和类型。

#### ④ 启动服务并验证真实监听

```
systemctl enable --now httpd.service
systemctl is-active httpd.service
systemctl is-enabled httpd.service
ss -lntp | grep ':82'
```

若服务仍失败，查看当前服务日志和近期 AVC，判断错误是否仍然与端口或文件访问有关。不要根据旧日志继续修改。

#### ⑤ 处理 firewalld 并验证功能

firewalld 的完整配置归第 21 章。此处只按题目所用 zone 做最小持久放行，并核对当前和永久状态。例如目标 zone 已确认后：

```
firewall-cmd --permanent --add-port=82/tcp
firewall-cmd --reload
firewall-cmd --list-ports
firewall-cmd --permanent --list-ports
```

本机功能验证：

```
curl http://localhost:82/
```

再从题目指定客户端验证，避免只证明回环路径。

#### ⑥ 证明配置不依赖临时标签

```
restorecon -Rv /srv/exam-site
matchpathcon -V /srv/exam-site/index.html
curl http://localhost:82/
```

最终记录：fcontext 本地规则、预期与当前标签、TCP 端口类型、服务 active/enabled、真实监听、firewalld 当前/永久规则和最终页面请求。只有这些证据共同满足，才能认为配置形成完整闭环。

#### 典型错误

- 把临时标签或机械新增当作持久配置：chcon 后只看 ls -Z，或 TCP 82 已有记录时仍重复 -a；
- 把局部状态或扩大授权当作完整功能：只看 systemctl status 或本机 curl，给静态站点整个目录可写类型，且页面成功后不再执行 restorecon 复验。

## [经典任务] 最小放行 Web 应用连接 MariaDB

---

### 环境

`httpd.service` 当前运行，站点页面能够正常加载。文件上下文、监听端口、`firewalld` 和数据库网络连通性已经由题面确认。应用需要连接题面指定的 MariaDB 服务；数据库地址、端口和凭据配置已核对。SELinux 为 Enforcing，重新触发请求后，近期拒绝证据指向 `httpd_t` 主动建立数据库网络连接的行为。

### 目标终态

- 从目标系统策略中查找并理解与 `httpd` 连接数据库相匹配的 Boolean；
- 只启用必要的精确开关；
- 修改在策略重新加载或系统重启后保持；
- 重新触发应用数据库操作并成功；
- 不启用范围更宽的无关网络连接开关，不生成本地策略模块。

## [参考解答] 最小放行 Web 应用连接 MariaDB

---

### ① 记录当前 Boolean 和业务证据

```
getsebool httpd_can_network_connect_db
semanage boolean -l | grep -i 'httpd.*database\|httpd_can_network_connect_db'
```

确认开关说明与“Web 应用主动连接数据库”一致。不要只因为名称包含 `httpd` 就启用。

### ② 建立持久终态

```
setsebool -P httpd_can_network_connect_db on
getsebool httpd_can_network_connect_db
```

等待命令完成后再查询，不要并发或重复运行多个 `-P` 操作。

### ③ 重新触发并分层验证

重新执行应用中需要数据库的具体操作。若仍失败，取得当前应用日志、服务日志和最新 AVC，再检查：

- 应用实际连接的主机和端口；
- 数据库服务是否可达；
- 防火墙和路由；
- 数据库账号与权限；
- 是否选择了错误的 Boolean；
- 是否还存在文件或端口类型问题。

Boolean 为 on 只是策略状态证据，最终验收是目标应用操作成功。

#### ④ 典型错误

- 直接启用范围更宽的 `httpd_can_network_connect`，但题目只要求数据库连接；
- 不读 Boolean 描述，仅凭网上示例复制；
- 不带 `-P`，只修改当前值；
- `setsebool -P` 尚未完成就重复执行；
- Boolean 已开启后不重新触发应用，只看命令返回码。

## [知识专题] 从考试操作迁移到真实变更和自动化

手工命令的价值不仅是完成一道题，还要形成可以被审计和自动化的状态模型。文件规则、端口映射和 Boolean 都具有清晰的“查询现状—声明终态—应用—验证”结构，适合后续由 Ansible 模块或 RHEL System Role 管理。但自动化不能替代前面的对象判断。

#### ① [工作迁移] 变更前保存本地定制基线

```
semanage fcontext -C -l
semanage port -C -l
semanage boolean -l
```

真实系统可能已经存在历史定制。先保存基线，确认目标规则是否被其他服务使用，再设计最小变更和回滚条件。

#### ② [工作迁移] 把“命令成功”改写成状态验收

自动化任务返回 `changed` 或命令退出码为 0，只证明工具执行成功。验收仍应包含：

```
声明状态存在
→ 当前内核或文件状态已应用
→ 服务重新加载了相关配置
→ 业务功能成功
→ 重启级持久性待验证
```

#### ③ [工作迁移] 保留可逆性和影响范围

删除或修改本地策略记录前记录旧值；对 `fcontext` 先预览递归变化；对端口确认同机服务冲突；对 Boolean 记录原值和业务理由。回滚不是简单执行相反命令，而是恢复原状态后重新验证受影响业务。

#### ④ [边界] 本章不展开自动化模块

Ansible 中管理 SELinux 文件规则、端口和 Boolean 的模块字段、collection 版本和幂等性属于 RHCE 自动化章节。本章只提供稳定手工对象模型，作为后续自动化的输入。

[Cheatsheet] 变更前保存本地定制；命令成功不是终态；回滚要恢复状态并重做业务验证。

## [本章收束] 用最小持久修改完成证据闭环

本章的核心不是背三组命令，而是始终先判断正在改变哪个对象。文件路径、协议端口和策略可选行为属于三个不同的策略入口；当前标签、持久规则和最终功能也属于三个不同的状态层。把它们混在一起，就会出现“标签看起来正确但没有规则”“端口类型正确但没有监听”“Boolean 已开启但业务仍失败”这类误判。

稳定的工作方法是：

- 确认失败层次
- 建立当前证据
  - 选择文件规则、端口类型或 Boolean
  - 做最窄的持久修改
  - 将声明应用到当前对象
  - 重新触发原始业务
  - 分层验证当前、持久与功能状态

### 主要判断表

| 现象或目标                           | 首要对象             | 下一条有区分度的证据                                      | 典型持久入口                                      | 不能单独证明       |
|---------------------------------|------------------|-------------------------------------------------|---------------------------------------------|--------------|
| 自定义目录无法被服务读取                    | 路径与文件类型          | <code>matchpathcon</code> 对比 <code>ls -Z</code> | <code>semanage fcontext + restorecon</code> | 服务已可用        |
| <code>restorecon</code> 后仍是错误类型 | fcontext 规则      | <code>semanage fcontext -C -l</code>            | 修改或删除冲突规则                                   | 业务根因已消失      |
| 服务不能绑定非标准端口                     | 协议、端口与 port type | <code>semanage port -l</code> 并核对协议             | <code>semanage port -a/-m</code>            | 进程已经监听       |
| 业务需要策略预留的可选行为                   | Boolean          | <code>semanage boolean -l</code> 阅读语义           | <code>setsebool -P</code>                   | 网络、账号或应用配置正确 |
| 页面当前能够访问                        | 完整访问链            | 再查持久规则并重做 <code>restorecon</code>               | 各层分别持久化                                     | 重启后仍成立       |
| 命令返回成功                          | 工具所管理的局部状态       | 查询真实状态并触发功能                                     | 依对象而定                                       | 整体终态正确       |

### 工作方法

- 调查只收集会改变决策的证据，不用重复执行失败命令代替分析。
- 新增前先查询；已有记录时区分修改、保留和删除，不无条件重建。
- 递归重标记前先预览范围；可写类型只授予真正需要写入的最小子目录。

- 每条验证命令都注明证明边界：规则存在不等于当前已应用，监听存在不等于远端可访问，Boolean 为 `on` 不等于应用功能成功。
- 本章没有连接 RHEL 9 虚拟机；命令和流程为课程、官方文档与手册页基础上的静态核对，重启级与端到端功能验证仍需在目标环境执行。

## 向下一章交接

下一章进入启动链、GRUB、Target 与系统恢复。本章只留下一个必要接口：`chcon` 之类的当前标签修改可能在恢复标签或系统重新标记时被覆盖，而持久 `fcontext` 规则描述路径未来应获得的类型。系统重新标记的启动与恢复流程归下一章；本章不提前展开 GRUB、救援目标或全系统 `relabel` 操作。